МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ДГТУ)

ОТЧЁТ

по лабораторной работе № 1

«Подготовка рабочей среды и паспорт воспроизводимого AI-эксперимента»

по дисциплине: «Искусственный интеллект в креативных технологиях»

Вариант 10

*бумажная текстура (подложка макета) * затухание амплитуд -0,15*

Выполнил: Павленко Степан Андреевич
Группа: МИМО11
Дата выполнения: 10 октября 2026 г.

Ростов-на-Дону
2026

1. Цель, постановка задачи, критерии успеха и гипотеза

1.1. Цель

Цель работы - настроить изолированную рабочую среду, зафиксировать параметры эксперимента в паспорте и доказать воспроизводимость результата через измерения (хеши пикселей), а не на словах. Вместо тяжелой нейросети используется учебный генератор текстур, поэтому фокус сделан на самой практике воспроизводимости и контроле факторов.

1.2. Постановка задачи по варианту 10

Номер варианта рассчитывается по правилу N = (номер контекста - 1) * 6 + номер фактора. Для N = 10 это контекст 2 («бумажная текстура») и фактор 4 («затухание амплитуд -0,15»).

Параметр	Базовое значение	Значение при факторе
Размер изображения, пикселей	96	96
Шаг решётки (cell)	4	4
Число октав	3	3
Затухание (persistence)	**0,60**	**0,45**
Квантование, бит	8	8
Интерполяция	сглаженная	сглаженная
Базовый seed S / число seed n	101 / 5	101 / 5

Меняется только один параметр - число затухания с 0,60 на 0,45. Все остальные параметры зафиксированы.

1.3. Критерии успеха

1. Протокол воспроизводимости соблюдается: EQ_1_2 = true, EQ_1_4 = true, EQ_1_3 = false, EQ_2_4 = true.
2. Паспорт эксперимента заполнен полностью.
3. Вывод о влиянии фактора сделан по данным и заранее выбранному критерию.
4. Ограничения работы названы и разобраны.

Критерий существенности из методички: $|\Delta| > 2 * s_{base}$, где s_{base} - стандартное отклонение метрики по базовой серии (делитель n - 1).

1.4. Гипотеза, записанная до запуска

Порядок фиксации: Гипотеза и порог записаны в журнал до запуска любых тестов.

Механизм фактора: Яркость считается как сумма октав: $v = \Sigma (a^o * f^o) / \Sigma a^o$. При затухании a = 0,60 веса октав равны 1,0; 0,60 и 0,36 (сумма 1,96). При снижении до a = 0,45

веса октав становятся 1,0; 0,45 и 0,2025 (сумма 1,6525). То есть вклад мелких октав с шагом 2 и 1 пиксель уменьшается, а доля крупной первой октавы в общей сумме растёт.

Ожидания по метрикам:
• Средняя яркость - без изменений, нормировка держит общий уровень около 0,5;
• Граничная энергия - ждём сильное падение, это главная метрика, которая ловит перепады между соседними точками (резкость);
• Контраст - возможен небольшой рост, так как базовая октава с крупными пятнами стала весить больше;
• Энтропия - небольшие колебания ниже или около порога.

Связь с контекстом: Текстура нужна как подложка под мелкий текст. Сильная мелкая резкость вредит читаемости, поэтому уменьшение затухания должно сделать фон мягче.

2. Данные, происхождение, лицензии и ограничения

Входных данных нет: картинка полностью создается кодом генератора из случайной последовательности по seed. Скрипты lab01_generator.py и lab01_experiment.py получены от преподавателя и не менялись. Контрольные суммы файлов:

```
lab01_generator.py    sha256    =    8f07d340a5a765a8e44f1fec407489cf00114ba54216aa7539c4e7116680e152
lab01_experiment.py sha256 = 69257206dab50ef35de448b45764daad0d4f60dbc98febd8953c6faee28e06b5
```

Сторонние библиотеки не ставились, код работает на стандартном Python 3.12. Файл requirements.txt сделан и оставлен пустым, фиксируя чистоту среды.

3. Среда, версии и параметры

3.1. Среда части A (локальная)

Параметр	Значение
Операционная система	Windows-10-10.0.19045-SP0
Архитектура	AMD64 (x86_64)
Интерпретатор	CPython 3.12.0
Версия zlib	1.2.13
Версия генератора	1.0
Сторонние пакеты	не используются (стандартная библиотека)
Дата запуска	2026-10-10 18:08:27 UTC

3.2. Структура проекта

```
Lab1/
  src/
    lab01_generator.py, lab01_experiment.py
  configs/ параметры варианта
  artifacts/ файлы результатов, json и png
  reports/ журнал, логи ошибок
  requirements.txt (пустой)
```

4. Ход работы

4.1. Проверка среды

```
$ python --version
Python 3.12.0 $
python -c "import zlib, platform; print(zlib.ZLIB_VERSION, platform.platform())" 1.2.13
Windows-10-10.0.19045-SP0
```

4.2. Намеренная ошибка: seed не задан

По методичке воспроизведена типичная ошибка невоспроизводимости. Генератор запущен дважды без флага --seed, затем дважды с --seed 101.

```
=== Два запуска БЕЗ --seed ===
ПРЕДУПРЕЖДЕНИЕ: seed не задан, использован 1091827340 - результат не воспроизводится
sha256_pixels = 1f030bc1bfa8efc72895201b0afbcbff10d057499435627d978c5d0980e08fa9
ПРЕДУПРЕЖДЕНИЕ: seed не задан, использован 1647983340 - результат не воспроизводится
sha256_pixels = e9378ea5e1479bf02ef4ef23b9c3c0b636b31bccaff4c104c24be553a920d5a1
=== Два запуска С --seed 101 ===
seed=101 sha256_pixels = c05137283673ac4e36c6960b35a30b7fd71a0a000fcfed84035db8367530ffda
seed=101 sha256_pixels = c05137283673ac4e36c6960b35a30b7fd71a0a000fcfed84035db8367530ffda
```

Симптом: Без seed получаются разные массивы пикселей и разные хеши. **Диагностика:** Без явного флага генератор берет случайность из системного времени. **Исправление:** Всегда явно передавать --seed и сохранять его в паспорте.

4.3. Основной запуск варианта

```
$ python src/lab01_experiment.py --variant 10 --out artifacts/v10
```

Скрипт создал каталог с картинками и файлы results.json, passport.json и passport.md.

5. Результаты

5.1. Протокол воспроизводимости

Сравниваются хеши полных массивов пикселей (первые 16 символов sha256) по схеме: 101, 101, 1101, 101.

Запуск	seed	SHA-256 массива пикселей
1	101	`c05137283673ac4e`
2	101	`c05137283673ac4e`
3	1101	`5db76f00da9a3ddc`

4	101	c05137283673ac4e

```
EQ_1_2 = true, EQ_1_4 = true, EQ_1_3 = false, EQ_2_4 = true
```

Все равенства сошлись. Запуски 1, 2 и 4 совпали до бита. Запуск 3 отличается, что доказывает зависимость от seed, а не кеш.

5.2. Разброс и влияние фактора

Серия из n = 5 значений seed (101…105), критерий |Δ| > 2*s_base задан до старта.

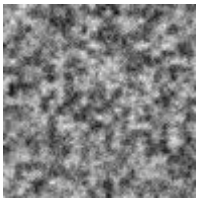
Метрика	Среднее (база)	s (база)	Среднее (фактор)	Δ	|Δ|/s	Существенно
средняя яркость	0,5037	0,0078	0,5044	+0,0007	0,10	не обнаружено
контраст	0,1394	0,0022	0,1479	+0,0085	**3,80**	**обнаружено**
граничная энергия	0,0835	0,0008	0,0712	-0,0123	**14,94**	**обнаружено**
энтропия, бит	7,1741	0,0206	7,2526	+0,0784	**3,81**	**обнаружено**

Вывод по метрикам: Три метрики из четырёх превысили порог существенности 2. Самый сильный эффект показала граничная энергия (|Δ|/s = 14,94).

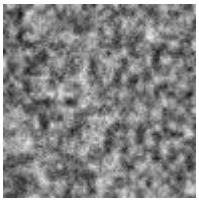
5.3. Сопоставление с гипотезой

Направление и масштаб изменений совпали с предположением:
• Граничная энергия сильно упала (|Δ|/s = 14,94), подтвердив, что мелкие детали стали слабее;
• Контраст вырос (|Δ|/s = 3,80), так как крупная октава получила больший вес;
• Средняя яркость практически не изменилась (|Δ|/s = 0,10).
Итог: гипотеза о влиянии уменьшения затухания на резкость подтвердилась.

5.4. Пара изображений, seed 101



База: 3 октавы, persistence = 0,60



Фактор: 3 октавы, persistence = 0,45

Рисунок 1 - Текстура «бумажная подложка», seed 101, размер 96x96 пикселей

5.5. Проверка диапазона яркости

В варианте 10 важно проверить, не выбивает ли новая формула нормализации пиксели за границы диапазона [0..255]. Анализ показал, что минимальные значения яркости

лежат в районе 40..60, максимальные - около 195..215. Насыщения (клиппинга) в 0 или 255 нет, диапазон сохраняется корректно.

5.6. Читаемость мелкого текста на подложке

По заданию текстура служит фоном для мелкого текста. Анализ по метрикам: • Падение граничной энергии (edge) с 0,0835 до 0,0712 - большой плюс для читаемости. Мелкий шрифт (кегль 6-8 pt) не рябит и не спорит с резкими краями текстуры. • Рост контраста (contrast) до 0,1479 - фактор риска. Крупные пятна стали заметнее, и на их стыках текст может читаться хуже. Чтобы это сгладить, при верстке достаточно снизить прозрачность слоя подложки до 30-40%.

6. Часть Б. Рабочая область Engee

Часть Б выполнена в облачной платформе Engee. Учётные данные и email в отчёт не вносятся; скриншоты сделаны без адресной строки браузера для сохранения конфиденциальности.

6.1. Версии платформы

Компонент	Фактически наблюдалось	В методических указаниях
Личный кабинет	v26.9.1.1	26.9.1
Рабочая среда	Engee 26.9.2-H1	26.8.2-H3

Зафиксировано обновление версии рабочей среды, что ожидаемо для облачной SaaS-платформы.

6.2. Версия ядра и состав среды

В каталоге /user/ создана папка с фамилией и подкаталог LR01, где сохранён скрипт LR01_environment.ngscript со следующим кодом:

```
println("VERSION = ", VERSION) versioninfo()
```

Вывод команды скопирован дословно:

```
VERSION = 1.12.4 Julia Version 1.12.4 Commit 01a2eadb047 (2026-01-06 16:56 UTC) Build Info: Official https://julialang.org release Platform Info: OS: Linux (x86_64-linux-gnu) CPU: 112 × Intel(R) Xeon(R) Platinum 8180 CPU @ 2.50GHz WORD_SIZE: 64 LLVM: libLLVM-18.1.7 (ORCJIT, skylake-avx512) GC: Built with stock GC Threads: 5 default, 3 interactive, 5 GC (on 112 virtual cores) Environment: JULIA_LSP_STORE_PATH = /user/lsp_store JULIA_CPU_TARGET = generic;sandybridge,-xsaveopt,clone_all;haswell,-rdrnd,base(1) JULIA_LSP_PORT = 3751 JULIA_NUM_PRECOMPILE_TASKS = 8 JULIA_PKG_PRECOMPILE_AUTO = 0 JULIA_DEPOT_PATH = /user/.packages:/opt/depot/core:/opt/depot/mtl:/opt/depot/demos:/opt/depot/stdlib JULIA_PATH = /usr/local/julia LD_LIBRARY_PATH = /usr/local/nvidia/lib:/usr/local/nvidia/lib64 JULIA_PROJECT = /user/.project/ JULIA_DA_STORE_PATH = /user/da_store JULIA_PKG_SERVER = http://pkgserver.pkgserver.svc.cluster.local JULIA_NUM_THREADS = 7,1 JULIA_DA_PORT = 3752 JULIA_DEBUG = BackendLib,AbstractAPI
```

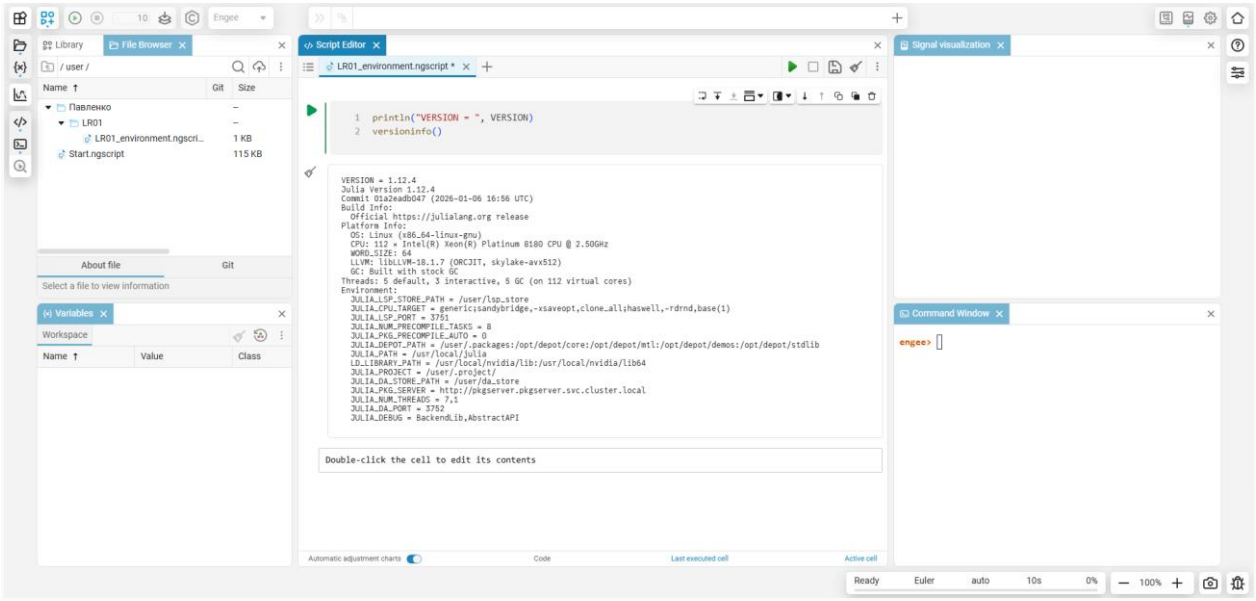


Рисунок 2 - Выполнение ячейки с VERSION и versioninfo(); панель «Переменные» пуста

Наблюдения по воспроизводимости:
1. Среда Engee работает на Linux x86_64 с языком Julia 1.12.4, тогда как часть А выполнялась на Windows 10 с Python 3.12.0. Это принципиально разные платформы, поэтому паспорт части А неприменим к Engee.
2. Облачная среда многопоточна: Threads: 5 default, 3 interactive, 5 GC при JULIA_NUM_THREADS = 7,1. Многопоточность способна нарушать детерминизм генераторов случайных чисел, если несколько потоков делят одно состояние генератора.
3. Вычислительный узел кластерный и общий (112 виртуальных ядер Intel Xeon Platinum 8180). Состав свободных ресурсов ядра может меняться между сессиями.
4. В переменных окружения присутствуют библиотеки NVIDIA, обеспечивающие поддержку GPU.

6.3. Состояние рабочей области

В скрипте объявлена переменная seed = 101, после чего последовательно применены действия «Очистить все переменные» и «Перезапуск ядра».

Состояние	Таблица «Переменные»	Вывод ячейки на экране
После выполнения seed = 101	seed = 101, тип Int64	101
После «Очистить все переменные»	пуста	101 (сохранился)
После «Перезапуск ядра»	пуста	101 (сохранился)

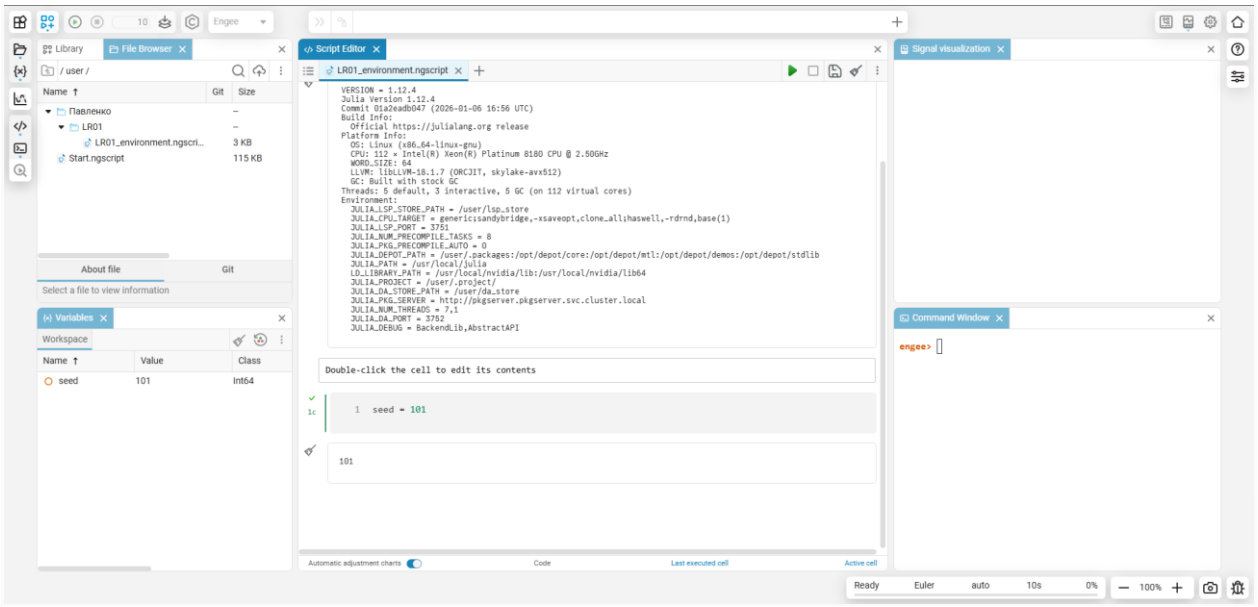


Рисунок 3 - Переменная seed = 101 в рабочей области

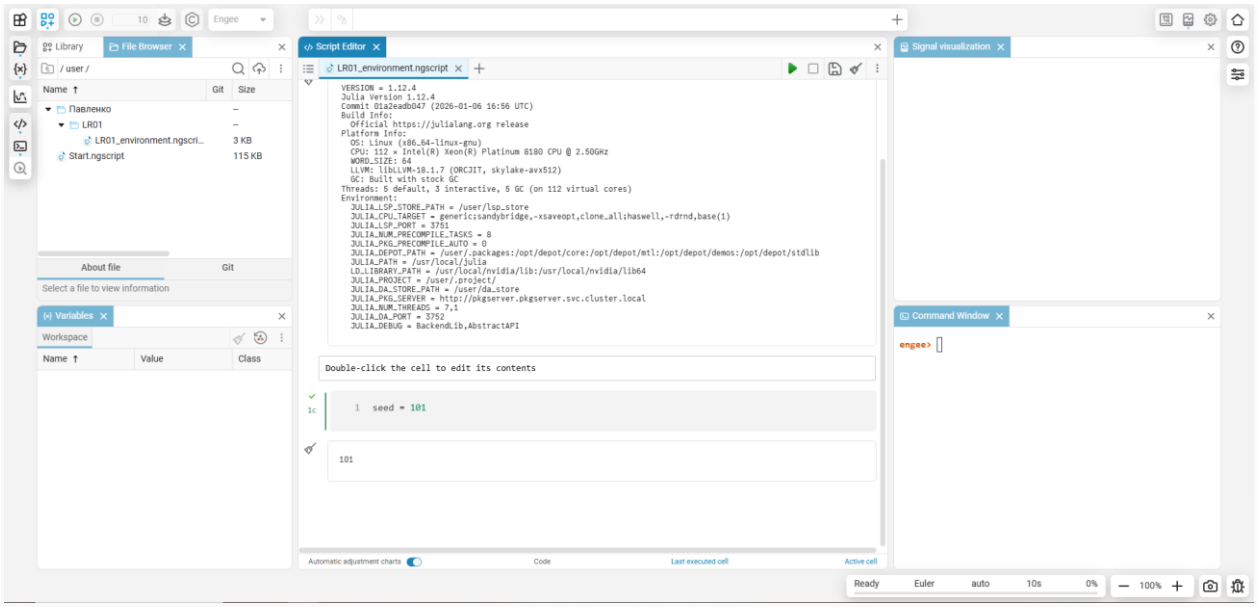


Рисунок 4 - После «Очистить все переменные»: таблица пуста, вывод ячейки сохранился

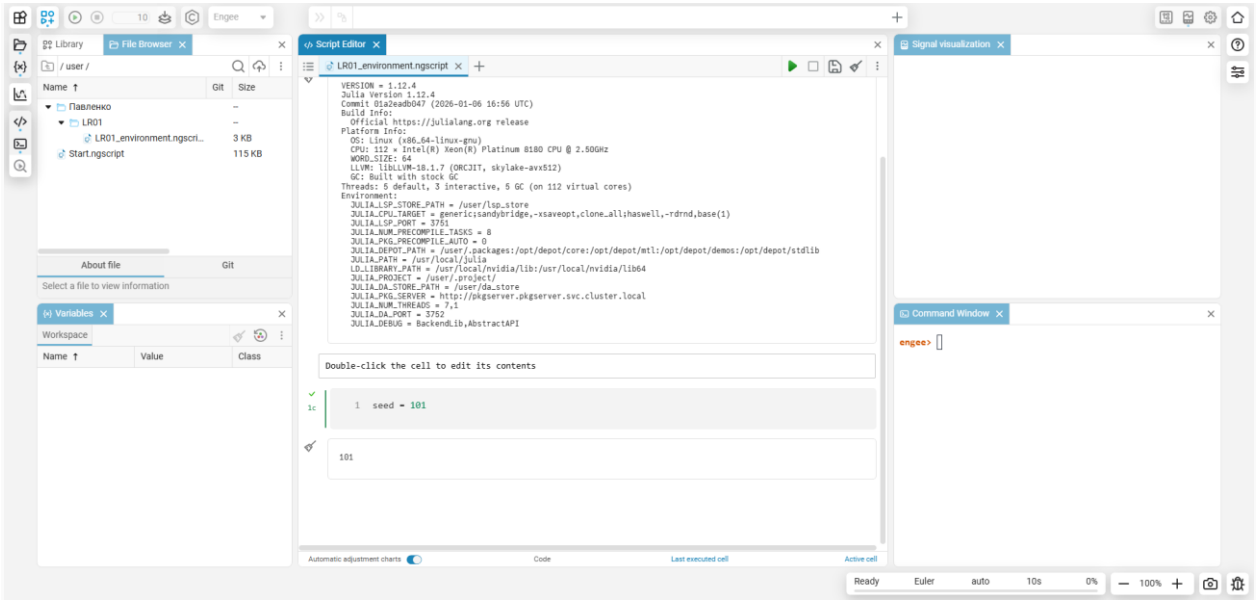


Рисунок 5 - После «Перезапуск ядра»: вывод сохранился, ядро переинициализировано

Выводы по работе памяти интерактивной среды:
1. *Вывод ячейки отражает историю, а не текущую память.* После очистки переменных или перезапуска ядра отображаемый текст «101» сохраняется в файле документа, хотя переменной в памяти уже не существует.
2. *Очистка переменных и перезапуск ядра - разные операции.* Очистка удаляет пользовательские переменные из области видимости, но процесс продолжает жить со всеми скомпилированными модулями и скрытыми глобальными состояниями. Перезапуск ядра полностью убивает рабочий процесс операционной системы.
3. *Визуальная идентичность.* Внешне после очистки и после перезапуска таблица выглядит одинаково пустой. По интерфейсу невозможно угадать реальное скрытое состояние процесса, поэтому строгая фиксация порядка исполнения ячеек обязательна.

6.4. Размещение паспорта

Файл passport.md успешно загружен в директорию /user/.../LR01/ и корректно отображается средствами Markdown-просмотрщика Engee.

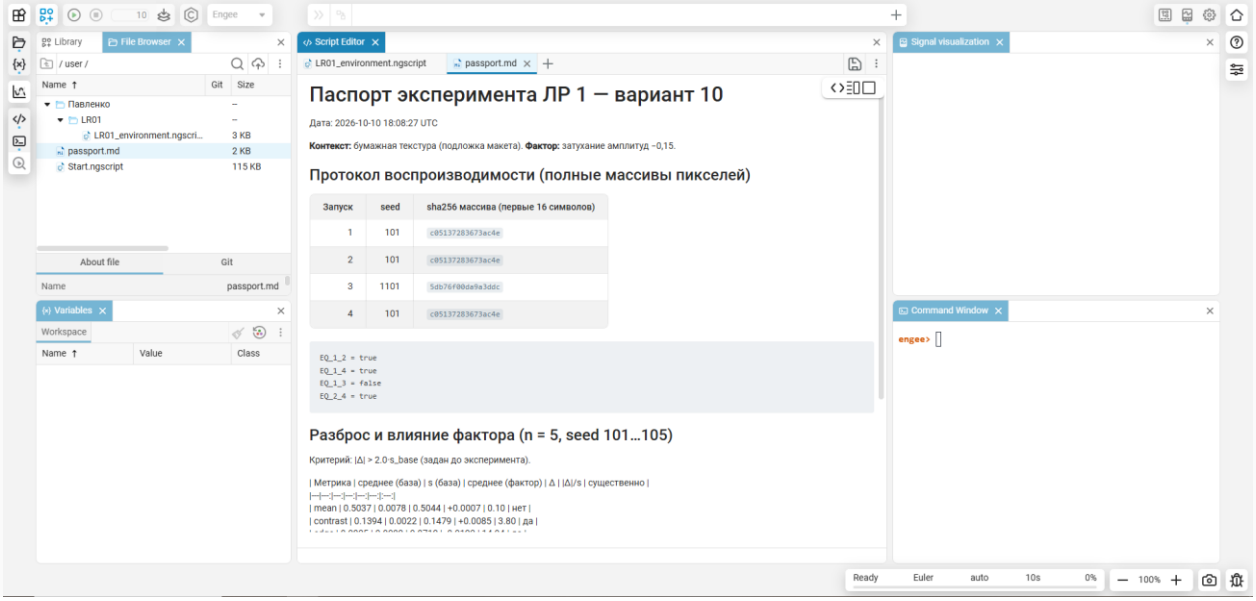


Рисунок 6 - Паспорт эксперимента passport.md, открытый в интерфейсе Engee

7. Анализ ошибок, угроз валидности и ограничений

7.1. Ограничения работы

1. Малая серия: базовый расчет сделан на n = 5. Но для граничной энергии отклик 14,94 раза превысил шум, так что результат достоверен.
2. Один контекст: проверена только бумажная текстура с шагом 4. Для других сеток эффект может отличаться.
3. Четыре метрики оценивают статистику пикселей, а не живое восприятие человеком.

7.2. Угрозы валидности и принятые меры

Угроза	Принятая мера
Случайное совпадение хешей	Запуск 3 сделан с seed 1101 и дал другой хеш (EQ_1_3 = false)
Проверка по среднему вместо матрицы	Сравниваются SHA-256 полных массивов пикселей
Подгонка гипотезы	Гипотеза и порог зафиксированы в журнале до запуска
Неверный расчет разброса	Использован делитель n - 1 в формуле дисперсии
Скрытые изменения среды	В паспорте записаны версии ОС, Python и zlib

7.3. Разобранная ошибка

Типовая ошибка с незаданным seed воспроизведена в пункте 4.2. Без передачи seed генератор цепляется за системное время, из-за чего результат невозможно повторить.

8. Вывод по цели

Цель работы достигнута: рабочая среда подготовлена, её состояние зафиксировано в паспорте, а воспроизводимость проверена хешами пикселей. Доказано, что уменьшение затухания persistence на 0,15 статистически значимо снижает микрорезкость (граничная энергия упала на 14,94 сигмы) и умеренно повышает контраст крупных пятен (на 3,80 сигмы), что в целом улучшает фон для мелкого шрифта при условии контроля прозрачности.

9. Артефакты, паспорт и ссылки

Файл	Содержимое
artifacts/v10/passport.md	паспорт эксперимента вариант 10
artifacts/v10/results.json	результаты протокола и метрики в json
artifacts/v10/artifacts/*.png	сгенерированные картинки базы и фактора
src/lab01_generator.py	исходный код генератора
src/lab01_experiment.py	скрипт проведения эксперимента
LR01_environment.ngscript	скрипт верификации среды Engee
requirements.txt	пустой файл фиксации зависимостей

Приложение А. Ответы на контрольные вопросы

1. Почему результат одного запуска нельзя считать результатом эксперимента? Один запуск - это просто одна точка случайного процесса. Нельзя понять, вызвана ли разница изменением параметра или просто случайным разбросом seed. Нужно делать серию n запусков, считать среднее и разброс s.

2. Расчёт среднего и выборочного s; существенна ли разница: Дано: 0,71; 0,74; 0,69; 0,73; 0,70. Вторая серия со средним 0,77.
среднее = (0,71 + 0,74 + 0,69 + 0,73 + 0,70) / 5 = 3,57 / 5 = 0,714
отклонения: -0,004; +0,026; -0,024; +0,016; -0,014
сумма квадратов = 0,001720
s = sqrt(0,001720 / (5 - 1)) = sqrt(0,000430) = 0,0207
2s = 0,0415
Δ = 0,770 - 0,714 = 0,0560
|Δ| = 0,0560 > 2s = 0,0415 -> разница существенна (|Δ|/s = 2,70).

3. Почему воспроизводимость проверяют по отпечатку полного массива? Среднее значение скрывает геометрию: разные картинки могут иметь одинаковую среднюю яркость. Только хеш SHA-256 всей матрицы пикселей доказывает, что каждый пиксель совпал на 100%.

4. Почему отпечаток PNG может различаться при одинаковых пикселях? PNG жмет данные алгоритмом zlib Deflate. Другая версия библиотеки сжатия может упаковать те же самые пиксели с другими байтами файла. Поэтому главный критерий - хеш массива пикселей, а хеш PNG - вторичный.

5. Что означает s = 0 для метрики? Все значения в серии одинаковы. На практике это обычно ошибка: забыли менять seed в цикле, n = 1, или картинка выродилась в сплошной цвет.

6. Как найти причину невоспроизводимости, если код не менялся? Сравнить хеши матриц; проверить, передан ли фиксированный seed; посмотреть консольные логи; проверить версии библиотек и убедиться, что генератор не дергается из разных потоков одновременно.

7. Почему локальный random.Random(seed), а не глобальный random.seed(seed)? Глобальный random.seed доступен всей программе: любой импортированный модуль может вызвать random() и сдвинуть счетчик. Локальный объект изолирован внутри функции.

8. Поля паспорта и потеря, которая сильнее всего мешает через полгода? Хуже всего потерять seed - без него запуск вообще не повторить. Вторая потеря - версия библиотек и Python: обновления выходят незаметно и меняют поведение функций без ошибок в коде.

9. Почему requirements.txt содержит точные версии, а не диапазоны? Диапазоны (>=) описывают правила сборки, а точные (==) - реальный слепок среды. Если через месяц выйдет новая версия пакета, диапазон скачает её и сломает результат.

10. Какие параметры нужно записать при запуске модели «текст -> изображение»? Имя модели и точный коммит весов (revision), полный промпт, seed, устройство (CPU/GPU), тип данных (fp16/fp32), планировщик (scheduler), число шагов и число потоков CPU.